

Fake Profile Detector

Consolidated Project Lifecycle Report (Steps 1-4)

Step 1: Define Scope & Requirements

1. Project Objectives: The primary objective of InstaGuard is to develop an automated machine learning tool to detect fake or bot-controlled Instagram profiles with an accuracy target of >90%. The tool assists users in verifying profile authenticity based on publicly accessible statistics, mitigating spam and fraudulent activity.

2. Core Features:

- **AI Training & Evaluation Pipeline:** Backend scripts framework to load datasets, engineer features, and train the model.
- **Web User Interface:** A simple web dashboard to input suspicious profile parameters and view predictions.
- **Dynamic Verdict Engine:** Provides instant friendly labels (*Looks Legit!* or *Looks like a Bot!*) backed by probabilities.

3. Constraints & Dependencies:

- Direct scraping is avoided to prevent Instagram API rate limits or IP bans; the system runs off user-supplied inputs.
- Relies on Python 3.x, Flask, Pandas, XGBoost, and Scikit-Learn. Must run locally without heavy hosting costs.

Step 2: System Design & Methodology

1. High-Level Architecture: The system utilizes an offline pipeline to train and save the XGBoost model to a serialized pickle file (*model.pkl*). The online inference layer loads this pre-trained model into memory via Flask, processing user string requests dynamically.

2. Feature Engineering Logic:

- **Follower-to-Following Ratio:** Captures follow-spammers who follow thousands of accounts but have minimal follower counts.
- **Posts-to-Following Ratio:** Flags bot patterns showing 0 active posts despite thousands of follows.
- **Username/Fullname Digit Ratios:** Computes alphanumeric density to detect randomly generated usernames.

Step 3: Development & Implementation (Engineering Logbook)

The codebase was modularized into logical modules inside the `src/` directory for loading, preprocessing, training, and inference. The layout is organized as follows:

```
d:\fake_profile_detector\  
■■■ app.py                # Flask web server and backend logic  
■■■ config.py             # Hyperparameter configuration  
■■■ main.py               # Master pipeline runner  
■■■ requirements.txt      # Python dependencies  
■■■ src/  
    ■■■ data_loader.py    # CSV file reader  
    ■■■ feature_engineer.py # Automated ratio computations  
    ■■■ preprocessor.py   # Train/test set splitting  
    ■■■ model_trainer.py  # XGBoost classifier training  
    ■■■ visualiser.py     # Model evaluation reporting  
    ■■■ predictor.py      # On-the-fly requests parser
```

Key Hurdles & Coding Solutions:

Hurdle A (Division-by-Zero): Accounts with 0 follows caused program crashes. A smoothing term (+1) was integrated: `df['#followers'] / (df['#followers'] + 1)`.

Hurdle B (Input Interface): Flask web users enter raw text, whereas models require numeric metrics. We wrote dynamic utility functions to compute digit-ratios, word counts, and bio-lengths in Flask endpoint requests on-the-fly.

Step 4: Testing & Data Collection

- 1. Dataset splits:** The model trained on 576 rows split 80% (460 instances) for training and 20% (116 instances) for evaluation.
- 2. Accuracy Scores:** Achieved **90.52% overall accuracy** using an XGBoost Classifier on unseen testing data.

Metric	Real Profiles (Class 0)	Fake Profiles (Class 1)	Weighted Avg
Precision	0.89	0.92	0.91
Recall	0.94	0.87	0.91
F1-Score	0.91	0.89	0.90
Support	63	53	116

- 3. Front-end Verification:** Dynamic test cases run through Flask server interface verified classifier boundaries:

Test Case	Profile Pic	Followers	Follows	Posts	Expected	Verdict	Status
1. Standard User	Yes	450	380	120	Real	Real	Passed
2. Follow Spammer	No	1	4500	0	Fake	Fake	Passed
3. Empty Bio Match	No	10	250	1	Fake	Fake	Passed
4. Balanced Private	Yes	150	180	45	Real	Real	Passed